

Zaman karmaşıklığı: Bir algoritmanın çalışma süresini ifade eder. Genellikle işlem sayısı ile doğru orantılıdır. Gösterimde genelde Big O kullanılır.

Hafıza karmaşıklığı: Bir algoritmanın çalışması sırasında ne kadar bellek kullanacağını ifade eder. Tanımlamalar, dizil kullanımının sürekli olması vb. etkiler.

* Bazı alg. lar sabit bellek kullanımı yaparken bazıları ise girdi boyutuna göre farklılaşan miktarlarda bellek kullanır.

Big O notasyonu: Bir alg. nın zaman veya hafıza karmaşıklığının büyük boyutlu problemlerde davranışını tanımlamak için kullanılır.

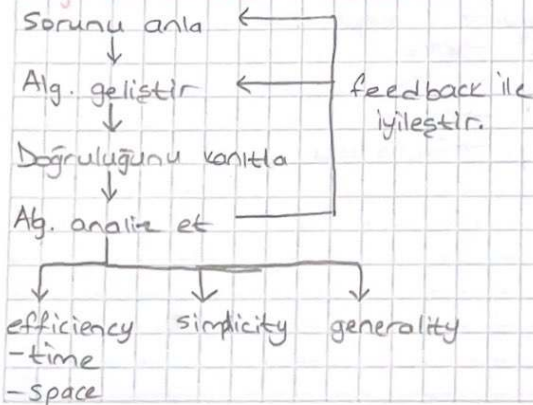
* En kötü durum senaryosunu formüle eder. $O(n)$ şeklinde gösterilir. "n" genelde girdi boyutudur. Sabit faktörler ve en yüksek dereceli terim hariçinde kalanlar ihmal edilir. Çünkü:
1- Bu yaklaşım büyük girdi boyutlarında davranış analizinde daha sık kullanılır. Çiğdan daha küçük terimlerin sonucu etkisi çok küçüktür.
2- Bu yaklaşım net bir hesaplama değil genel bir bakışla çıkarım yapmak için kullanılır.

Best case: Bir alg. nın en iyi ihtimaldeki karmaşıklığını ifade eder.

worst case: Best case 'in tam zıttıdır.

Big O notasyonu bu durumu gösterir.

Alg. ve Problem Tanımı:



Sieve Alg.: 2'den n'e kadar olan asal sayılar. $N=25$

1. yol → Her sayının kareköküne kadar olan sayılardan birine bölünüp bölünmeme kontrolü.

2. yol → Katları olan sayıları eleme (5'e kadar $\sqrt{25}=5$)

Sieve Alg.

Alg.: Bellirli problemi çözmek için kesin direktiflerden oluşan sonlu kümedir ve tüm girdiler için doğru sonucu üretir.

Alg. özellikleri:

Unambiguous: (Anlamsızlık) Adımları, giriş ve çıkışlar net olmalı.

Finiteness: sonluluk

Feasibility: mevcut imkanlarla yapılabilir olmalı.

Independent: Her hangi bir prog. dilinden bağımsızlık

Algoritmik Problemler:

- Sıralama
- Arama
- Kombinatoryal prob.
- Numerik prob.
- Metin işleme
- grafi problemleri
- geometrik prob.

Veri Yapıları:

Alg. ların erişme, değiştirme gibi işlemlerinde kullanılmak üzere bilginin saklandığı oluşuma veri yapıları denir.

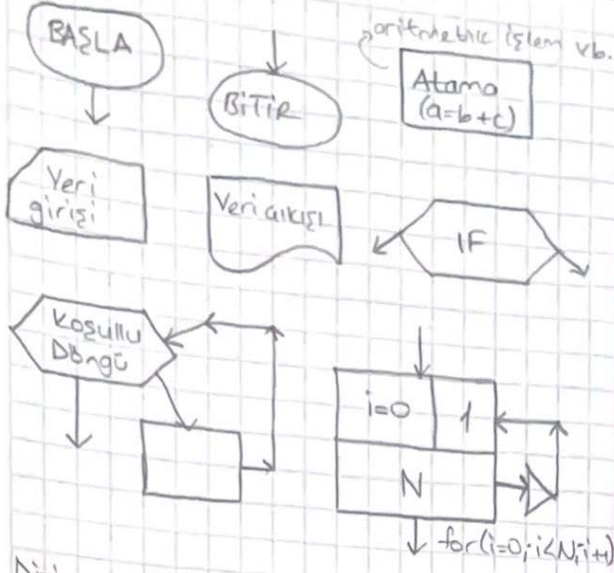
- Listeler → Diziler
- Queue
- Heap, öncelikli queue
- Stack
- Hash tabloları
- Ağaçlar
- Küme ve sözlük

Sözde Kod:

* Alg. nın bir dile bağlı olmadığını göstermekte etkilidir.

* Açık bir şekilde ne yapılması ve ne döndürülmesi gerektiği belirtilir.

Akış Diyagramı:



Dizi okuma → for içinde girdi alımı

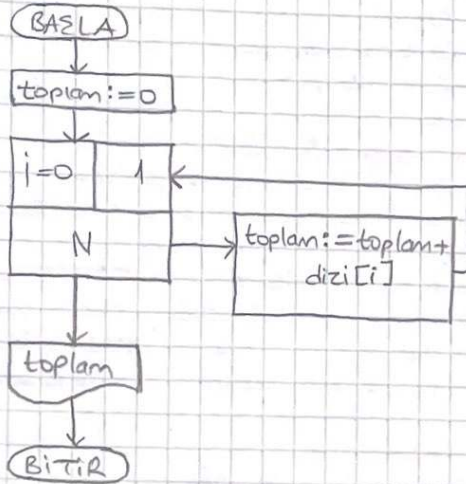
Veri girişi bloğu içinde $N, A(N)$ tanımlı
örnekleme

* oklar köşeli olmalı, eğri olmamalı.

* Dile özel operatörler olmamalı. ($a++$ vb.)

* IF 'te T/F kolları belirtilmeli. T mutlak olmalı, F şart değil.

ÖR: Bir dizinin elemanları toplamı



Algoritma Analizi:

- Alg. performansını ölçmek için
- Farklı alg. ile karşılaştırmak için
- Değerlendirmeyi araştırmak için analiz yapılır.

* İki tür analiz vardır:

- Teorik analiz
- Empirik analiz

Empirik analiz: Alg. nın bir prog. dili ile çalıştırılarak işletim süresinin karşılaştırılmasıdır.

* sistemin ve dilin özelliklerine göre bu yöntemde sorunlar çıkabilmektedir.

* Alg. başarımının:

- bilgisayar özelliklerinden
- kullanılan veri "
- prog. dili avantaj/dezavantajlarından bağımsız ölçülmesi gerekir.

Teorik analiz: Yukarıda bahsedilen bağımsızlık şartları ile analiz yapan analiz türüdür.

* Öncelikle önemli işlemlerin sayısı belirlenir. Sonrasında büyüme fonk. ile alg. etkinliği ifade edilir.

* Alg. nın oransal zaman gereksinimi büyüme oranı olarak isimlendirilir. Alg. başarımının göstergesidir.

- C → sabit
- $\log N$ → logaritmik
- $\log^2 N$ → " kare
- N → doğrusal
- $N \log N$ → " logaritmik
- N^2 → kare
- N^3 → kübik
- 2^N → üstel
- $N!$ → faktöriyel

Big O notasyonu:

* Bir alg. $g(n)$ ile doğru orantılı zaman gerektiriyorsa $g(n)$ sergisindedir denir ve $O(g(n))$ ile gösterilir.

$O(\log n)$ $O(n^2)$ $O(n \log n)$ vb.

Özyinelemeli Alg. Big O:

* $T(n)$ yineleme denklemi olarak adlandırılır.

* Özyinelemeli fonk. Big O'su bulunabilmesi için yineleme ilişkisini çözmek gerekir.

Selection sort:

* Her iterasyonda en küçük (en büyük) eleman seçilir. Baştan sona yerleştirilir.

$O(N^2)$

Bubble Sort:

* Elementer baştan sona iki'li karşılaştırılır. Büyük olan sona taşınır.

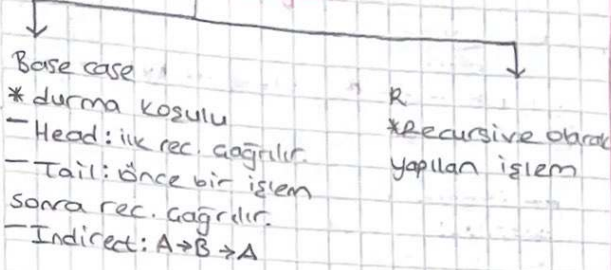
$O(N^2)$

Insertion Sort:

* İkinci elemandan başlayarak kendinden öncekilere karşılaştırılır. Önceki eleman büyükse yer değiştirilir. Değilse sıradaki elemene geçer.

$O(N^2)$

Recursive Fonksiyon:



Recursive

- Small code size
- Overhead: fonk. → fonk. geçerken eski bilgilerin geri yüklenmesidir. (malîyet)
- Memory (overhead yükünden)

Iterative

- Large code size
- No overhead
- No extra memory needed

Ör: `int fib(int n){`
`if(n==1){`
`return n;`
`}`
`return fib(n-1)+fib(n-2);`
`}`

`int fibIte(int n){`
`int i;`
`int current, previous=1, prepre=0;`
`if(n==0){`
`return 0;`
`}`
`for(i=2; i<n; i++){`
`current=previous+prepre;`
`prepre=previous;`
`previous=current;`
`}`
`return previous;`
`}`

Backtracking: Labirent örneği gibi belirli yollar deneyip geri dönmek istediğimizde kullanılır.

* Recursive'in avantajıdır.

Ör: `5 0 0 0`
`1 1 1 1`
`0 1 1 0`
`0 0 1 1`

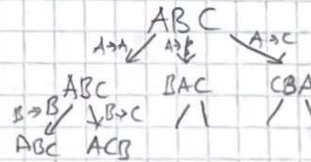
`int isValid(int x, int y, int **maze, int n){`
`return maze[x][y] == 1 && 0 <= x && x < n`
`&& 0 <= y && y < n;`
`}`

`char* findPath(int x, int y, int **maze,`
`int n, char* currentPath){`

`int i;`
`int nextX, nextY;`
`if(x==n-1 && y==n-1){`
`return currentPath;`
`}`
`maze[x][y]=0;`
`for(i=0; i<4; i++){`
`nextX = findNextX(x, i);`
`nextY = findNextY(y, i);`
`if(isValid(...)){`
`addCurrentPath(currentPath, i);`
`findPath(nextX, nextY, ...);`
`removeCurrentPath(currentPath);`
`}`
`return currentPath;`
`}`

Ör: Harf permutasyonu

ABC ACB
 BCA BAC
 CBA CAB



`void Permutate(char* str, int left, int`
`int i;`
`if(left == right)`

Telefondan
 girer.

Rec. Ürnek Göz.

ÖR: P1 P2

5 7 8 11 2

oyunu 1'in
kazanma durum
ları?

Linkli Listeler:

typedef struct NODE {

int data;

struct NODE * next; struct NODE * prev;

} node;

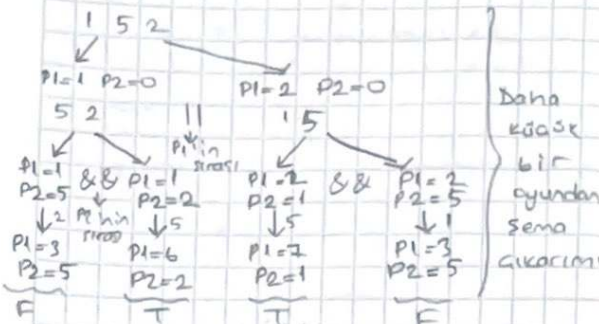
- single : sadece next bulunur.

- double : next ve previous

- circular : dairesel

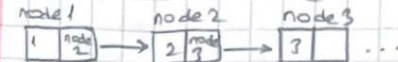
Head → Node ... → tail next = Null;

NOT: sürekli son node'un next'ini Null yap-
mak gerek.



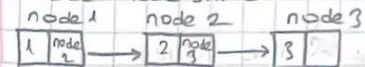
```
int playGame (int * nums, int playerTurn,
int resBegin, int resEnd, int firstScore, int secondScore)
{
    int arrayLength = sizeof(nums)/sizeof(int);
    if (arrayLength == 0) {
        if (firstScore >= secondScore)
            return 1;
        else
            return 0;
    }
    if (playerTurn == 0) {
        // oyun planının kopyası değiştirilir.
        resBegin = playGame (removeFirst(nums), 1,
        firstScore + nums[0],
        secondScore);
        resEnd = playGame (removeLast(nums), 1,
        firstScore + nums[arrayLength-1],
        secondScore);
    }
    if (resBegin || resEnd)
        return 1;
    else
        return 0;
}
else {
    resBegin = playGame (1 yerine 0, second
    Score artacak);
    resEnd = playGame (" " " ");
    if (resBegin && resEnd)
        return 1;
    else
        return 0;
}
```

Insertion: node ekleme

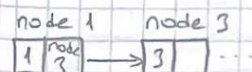


Önce node 2'nin adresi node 4'ün
next'ine gelir.
Sonra node 1'in next'i node 4'ün
adresine olur.

Delete: node silme



silme için



Search: single ise baştan sona, double ise
hem baştan hem sondan başlanabi-
lir.

Avantajlar:

- 1-Ekleme - çıkarma daha kolay yapılabilir.
- 2-Pespeşe yer tutmamıza gerek kalmıyor.
- 3-Ekleme - çıkarmada kaydırma gerek kalmıyor.

1-Dynamic Memory

2-Effective RAM management

3-Easy ins & del

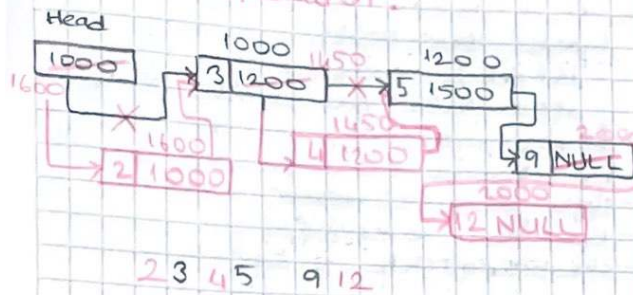
Dezavantajlar:

1-Random Access yapılamıyor.

2-Extra memory ister. (pointerları tutmak vb.)

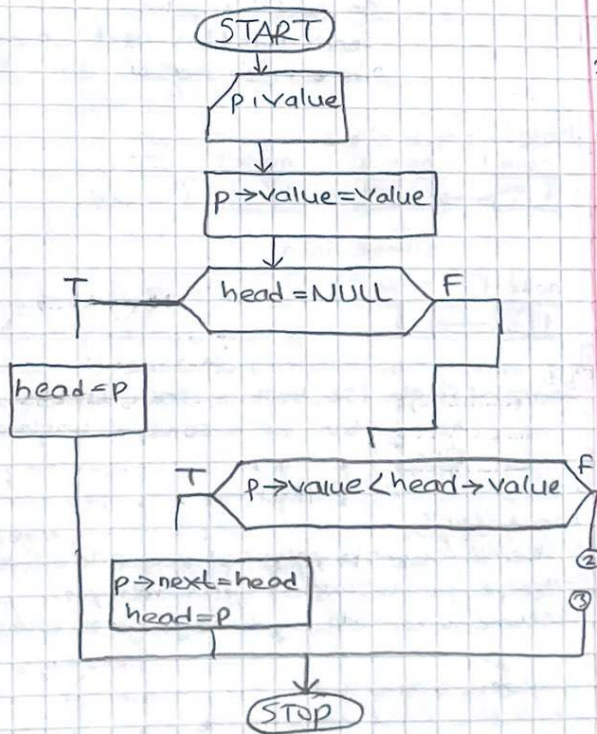
* Array'lerde malloc vb. ile yer açılmıyorsa
Compile time'da yer açılır. Linkli liste run
time'da yer açar.

insertion operation:

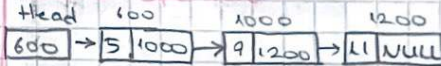


```

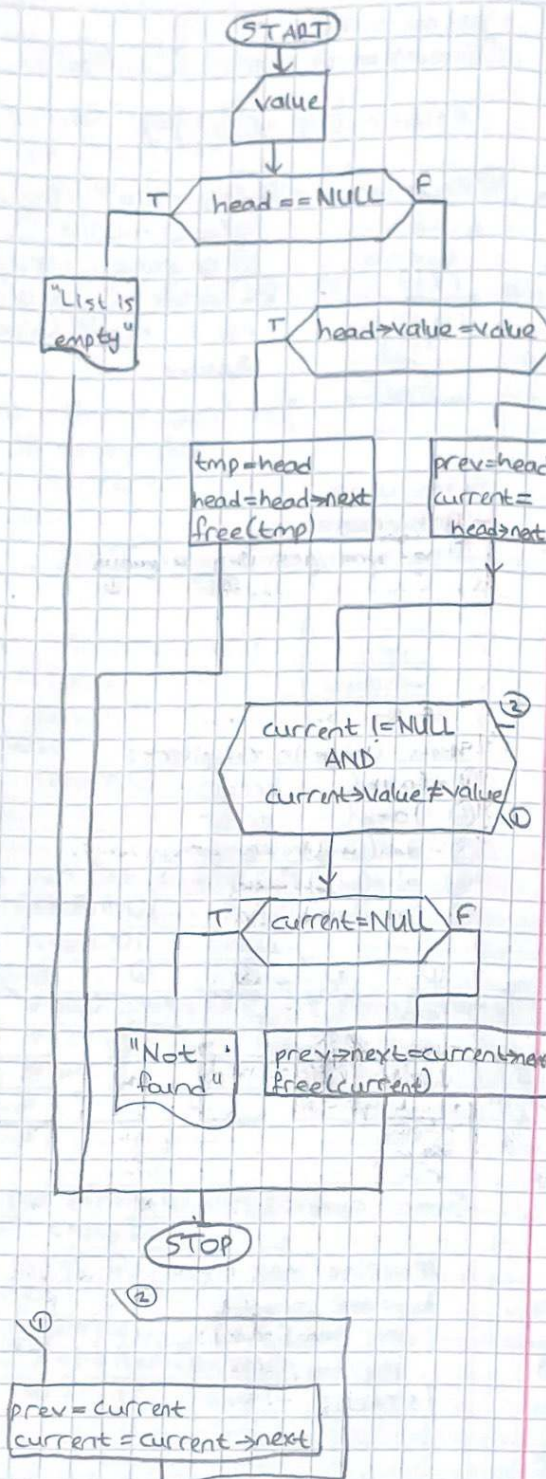
struct NODE *createNode() {
    struct NODE *p;
    p = (struct NODE *) malloc (sizeof(NODE));
    if (p == NULL) {
        printf("Allocation error");
        exit(0);
    }
    return p;
}
    
```



Delete Operation:



- 1- Liste boş olabilir mi?
- 2- Baştaki elemanı sileriz.
- 3- Ortadaki "
- 4- Sontaki "
- 5- Eleman listemizde olmayabilir.

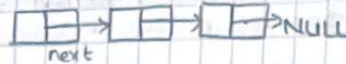


Linked List:

- Structure
 - Arrays
 - matrix
- gösterim şekilleri

Linked List Types:

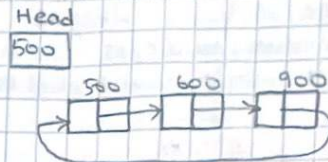
① Single Linked List



② Double Linked List



③ Circular Linked List



④ Circular Doubly Linked List



Advantages

- Dynamic Memory Structure (No need to give initial size)
- No Memory Wastage
- Insert and Deletion at times (search) Operations
- Flexible (No contiguous Memory)
- Efficient For large data
- Scalability

Disadvantages

- Memory Usage
- Traversal
- Reverse Traversing
- Random Access
- Lower efficiency
- Complex implementation
- Difficult to share
- Not suited for small dataset
- Cache inefficiency

Applications:

- To implement stack and queue
- For representation of trees and graphs
- Dynamic memory allocation
- Representing sparse matrices
- For performing arithmetic operations on long integers
- For finding paths in networks
- Memory management, process scheduling, file system
- For algorithms that need to frequently insert or delete items

Real-Life Applications:

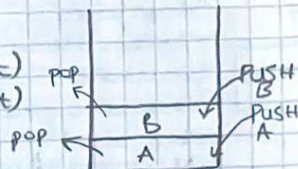
- Music Player
- Web browser
- Image Viewer
- Switching between two apps
- Saving contact info in mobile phones

Circular Linked List Implementations

- Implementation of a queue
- Music or media player
- Hash table implementation
- Navigation systems
- Task Scheduling

STACK (Yığın):

- Abstract data type
- FILO (First in last out)
- LIFO (last in first out)



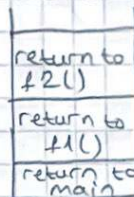
push()
 pop()
 peek() } en üstteki karakteri
 top() } görmek için
 isFull()
 isEmpty()
 size()

Applications:

① Evaluation of arithmetic operations

$$A + (B + C - (D/E * F) * G) * H$$

② Recursion



* ③ Reversing String

④ Back tracking

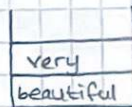
* ⑤ Parenthesis matching

* ⑥ Conversion of decimal to other number systems

Stack Usage:

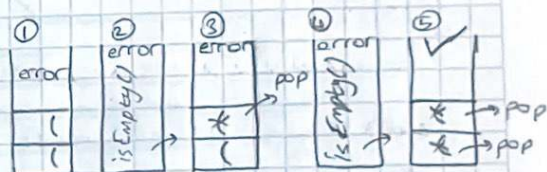
- Undo/Redo

The weather is very beautiful.



Stack Usage in Compilers:

- ① -(a+b(error
- ②)a+b(error
- ③ a*(b+c(error
- ④ a)*(b+c) error
- ⑤ ((a+b)*c) ✓



Stack operations:

```

#define MAX 10
typedef struct {
    int item[MAX]
    int top;
} STACK;
  
```


#define MAX 10

```
void initStack(STACK *s){
    s->top = 0;
}
```

```
int isFull(STACK *s){
    if(s->top == MAX)
        return 1;
    else
        return 0;
}
```

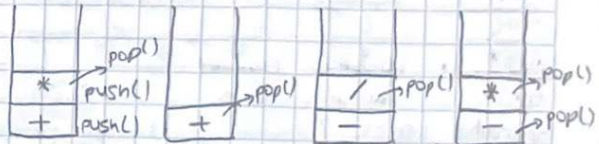
```
int pop (STACK *s, int *x){
    if (is Empty(s))
        return 0;
    else{
        --s->top;
        *x = s->item[s->top];
        return 1;
    }
}
```

Diagram illustrating the push operation on a stack. The stack initially contains element 5. The sequence of operations shown is:

- Initial State:** Stack contains 5. top points to the bottom.
- Operation 1:** $push(5)$. Element 5 is added. top moves up.
- Operation 2:** $push(7)$. Element 7 is added. top moves up.
- Operation 3:** pop . Element 7 is removed. top moves down.
- Final State:** Stack contains 5. top points to the bottom.

Arithmetic Expression Evaluation

Precedence of Arithmetic Operations:

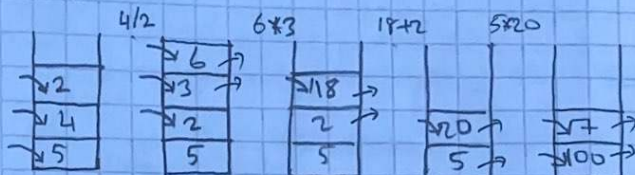
$$a + b * c - d / e * f$$


operand

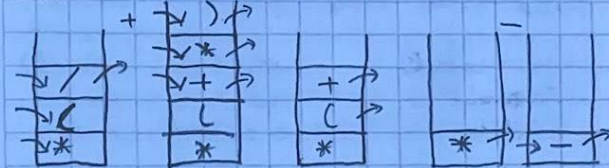
Aithmetic Operation on Compilers in RealLife

$$X = 5 * (4/2 + 3 * 6) - 7$$

operand
stack



operator
stack

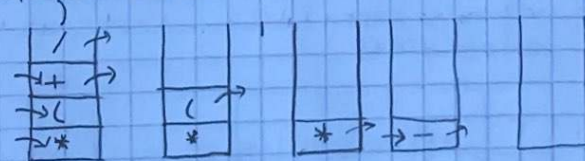


Precedence :

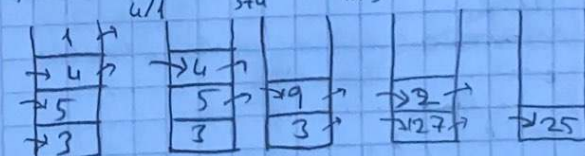
Push	Pop	
(	*/	Highest
*/	+ -	
+ -	)	

$3 * (5 + 4/1) - 2$ infix
 $3541/+*2-$ postfix

operator stack



operand stack



Applications of Stack data structure :

- Function call and recursion
- Undo/Redo operations
- Expression Evaluation
- Browser history
- Balanced parenthesis
- Backtracking algorithm

Real-time Applications :

- Stack of books in a bookshop
- Text editors
- Web browsers
- Call logs
- E-mail
- Photos
- Notifications

Advantage of Stacks :

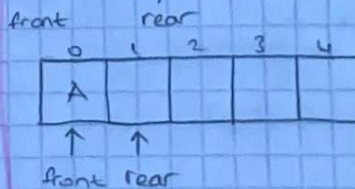
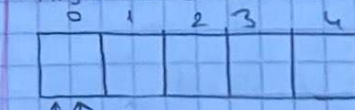
- Easy implementation
- Fast access time
- Helps in function calls
- Supports backtracking
- Used in compiler design
- Enables undo/redo operations

Disadvantages of Stacks :

- Limited capacity
- No random access
- Not suitable for certain applications (sort and search)
- Stack over flow and under flow

Queue (Kuyruk) Data Structure (Abstract)

*First in First out



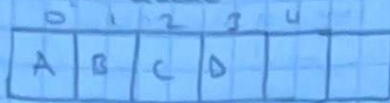
- Circular Queue

- priority Queue

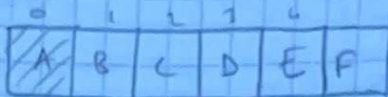


- Double ended queue

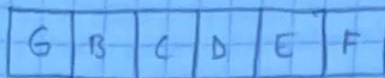
Circular Queue



↑ Front ↑ rear



↑ Front ↑ rear



↑ rear ↑ Front

Queue Operations:

- is Empty()
- is Full()
- enqueue()
- dequeue()
- initQueue()

```
int isEmpty() {
    if (front == -1)
        return 1;
    else
        return 0;
}
```

Circular Queue

```
int isFull() {
    if (rear == MAX-1)
        return 1;
    else
        return 0;
}
```

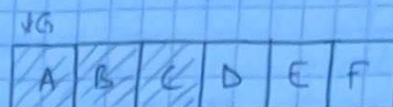
```
int isEmpty() {
    if (size == 0)
        return 1;
    else
        return 0;
}
```

How circular Queue

```
int isFull() {
    if (size == MAX-1)
        return 1;
    else
        return 0;
}
```

```
int dequeue(int *x) {
    if (isEmpty())
        return 0;
    else {
        *x = q[front++];
        size = size - 1;
    }
    return 1;
}
```

```
int enqueue(int val) {
    if (isFull())
        return 0;
    else {
        rear++;
        q[rear] = val;
        size = size + 1;
    }
    return 1;
}
```



↑ rear ↑ Front

Dequeue A
" B
Enqueue E
" F
Dequeue C
Enqueue G

```
void initQueue() {
    int front = -1, rear = -1;
}

int isEmpty() {
    if (front == -1)
        return 1;
    return 0;
}

int isFull() {
    if ((front == rear + 1) || ((front == 0) && (rear == MAX-1)))
        return 1;
    return 0;
}
```

```
int enqueue(int element) {
    if (isFull())
        return 0;
    else {
        if (front == -1)
            front = 0;
        rear = (rear + 1) % SIZE;
        q[rear] = element;
    }
    return 1;
}
```

```

    (c)
    int dequeue (int *x){
        if (isEmpty())
            return 0;
        else{
            *x = a[front];
            if (front == rear){
                front = -1;
                rear = -1;
            }
            else{
                front = (front + 1) % SIZE;
            }
            return 1;
        }
    }

```

NOT: 2 stack ile 1 queue, 1 queue ile bir stack oluşturulabilir.

Comparison Stack vs. Queue:

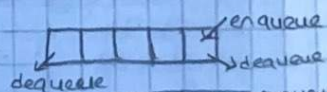
	stack	Queue
working principle	LIFO	FIFO
structure	Same end is used to insert and delete	One end is used for insert, other end is used for delete
Number of pointers	one	two
Operations	push and pop	Enqueue, dequeue
Variants	No variant stack	circular,
Implementation	Simple	Comparatively complex

Types of Queues:

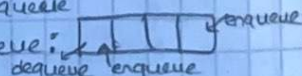
1. Circular queue:



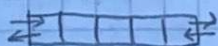
2. Input restricted queue:



3. Output restricted queue:



4. Double ended queue:



5. Priority queue:

Linear Data Structures:

- Array
- Stack
- Linked List
- Queue

Non-Linear Data Structures:

- Graphs
- Tree (Hierarchical yapısı)

Non-Linear Data Structure

* The data items are arranged in an order such as geographical coordinates, rectangles or polygons (kd tree) manner where the elements are attached adjacently.

* It arranges the data in a sorted order and there exists a relationship between the data elements.

Ease of implementation:

- Levels involved
- Memory utilization

Simpler: Linear

- Single level
- ineffective

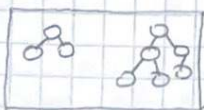
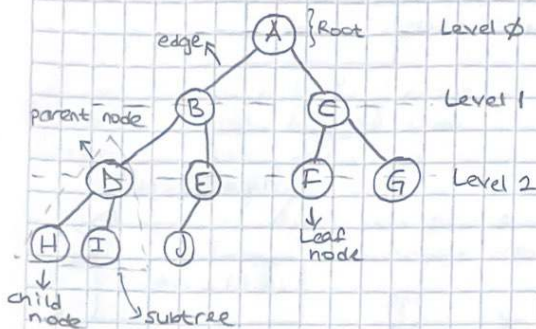
Complex: Non/linear

- Multiple level
- effective

Tree

- Path
- Root
- Parent - Ancestor
- Child
- Leaf

- Subtree
- Visiting
- Traversing
- Level
- Forest



Forest

Visiting: Döğme
uğrama
Traversing: Ağacı
gezinme

Tree Applications in Real-Life:

- File management
- Compression Algorithms (Huffman Coding)
- Databases
- Compilers (Syntax Tree)
- Priority Queue
- AI (Decision Tree, Random Tree)
- Indexing multi dimensional information
- such as geographical coordinates, rectangles or polygons (kd tree)

Types of Tree:

- Binary Tree
- Binary Search Tree
- AVL tree
- B-tree
- R-Tree
- Interval Tree
- D-Ary Tree
- Heap Tree

Tree Traversal:

- 5-15-8-7-9
from top to down left to right
- 7-9-15-8-5
from down to top left to right

1. In order traversal Left-node-right
2. Pre order " node-left-right
3. Post " " Left-right-node

In order traversal

7-15-9-5-8

pre order traversal

5-15-7-9-8

post order traversal

7-9-15-8-5

struct node {

```
int data;
struct node *leftc;
struct node *rightc;
```

void inOrderTraversal (struct node *root){

```
if (root == NULL) return;
inOrderTraversal (root->left);
printf ("%d", root->data);
inOrderTraversal (root->right);
```

}


```

void preOrderTraversal(struct node *root){
    if(root == NULL) return;
    printf("%d", root->data);
    preOrderTraversal(root->left);
    preOrderTraversal(root->right);
}

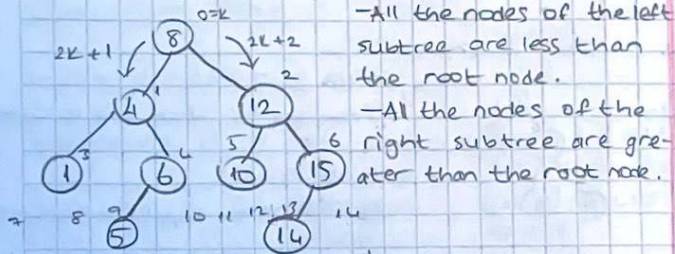
```

```

void postOrderTraversal(struct node *root){
    if(root == NULL) return;
    postOrderTraversal(root->left);
    postOrderTraversal(root->right);
    printf("%d", root->data);
}

```

BST (Binary search Tree)

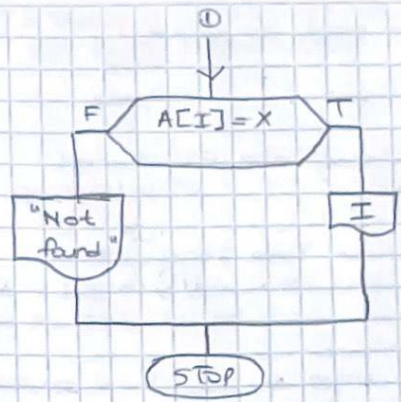
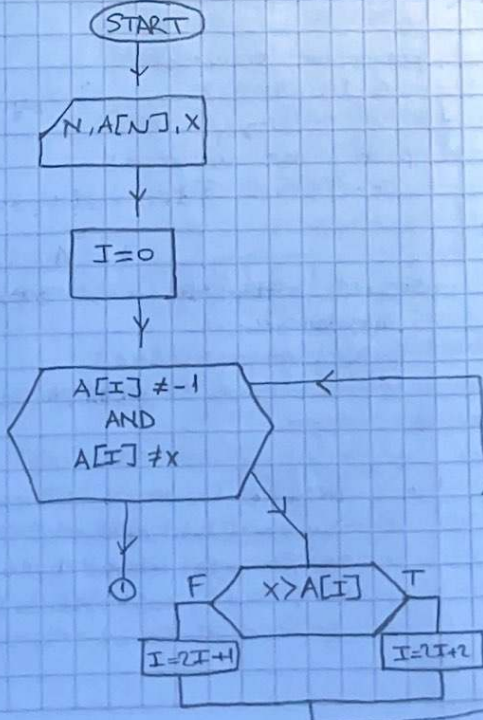


-All the nodes of the left subtree are less than the root node.
-All the nodes of the right subtree are greater than the root node.

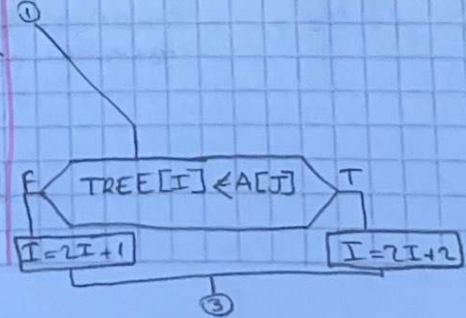
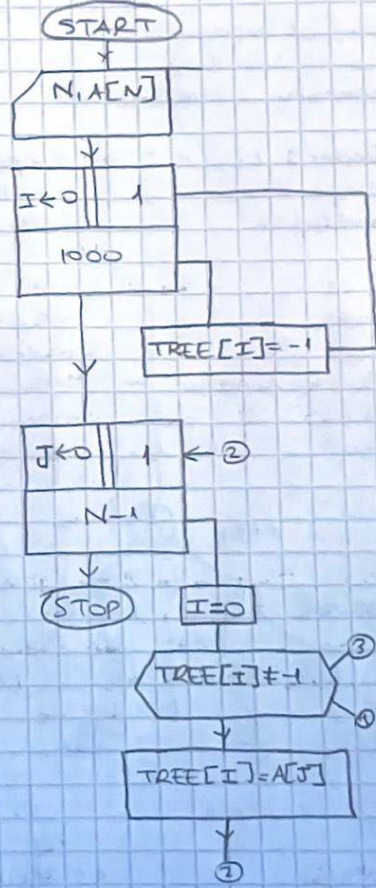
diğer implementasyonları
-Search -insert -delete

*Elementler üzerinde işlem yaparken her elementin unique olduğu varsayılır.

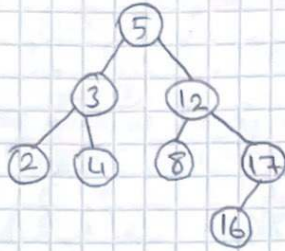
Search:



Insert:



Delete Operation on BST:



Case I: Leaf node

Case II: Internal node with a single child

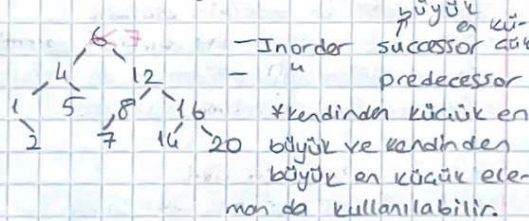
Case III: Internal node with two child

Case IV: Not found

BST complexities:

Operation	Average Case	Worst Case
Search	$O(\log N)$	$O(N)$
Insert	$O(\log N)$	$O(N)$
Delete	$O(\log N)$	$O(N)$

Space Complexity $\rightarrow O(N)$



Types of Trees:

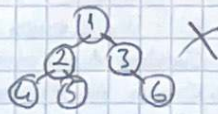
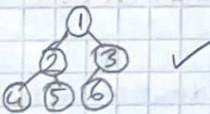
• Binary Tree $\rightarrow$ En fazla iki çocuk

• Ternary Tree $\rightarrow$ En fazla üç çocuk

• N-ary $\rightarrow$ N adet çocuk

• Complete Binary Tree

$\rightarrow$ Son seviyenin üstü tam dolu, son seviyede sola dayalı doldurulmuş ağac.



• Full Binary Tree

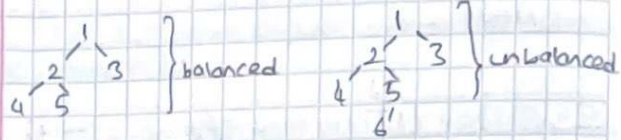
$\rightarrow$ Her düğümün ya hiç çocuğu olmayacak ya da 2 çocuğu olacak

• Perfect Binary Tree

$\rightarrow$ Tüm ara düğümlerin 2 çocuğu olmalı

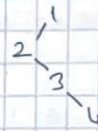
• Balanced Binary Tree

$\rightarrow$ Ağacın sol ve sağ leveleri arası fark 0 ya da 1 olmalı.



• Degenerated or Pathological Tree

$\rightarrow$ Kesin bir yatma yönü olmayan ama tek çizgi ile ilerleyen ağac



• Skewed Binary Tree



• AVL-Tree (self-balancing Tree)

• Red-Black Tree

• B-Tree (self-balancing Tree)

• B+ Tree (File systems and Databases)

Ağacın derinliği nasıl bulunur?

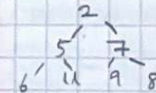
Heap Tree:

• Max-Heap Tree: Parent'ın değeri çocuklardan büyük olmalı



$$A[\text{parent}(i)] \geq A[i]$$

• Min-Heap Tree: Parent'ın değeri çocuklardan küçük olmalı



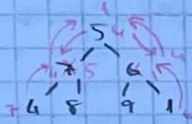
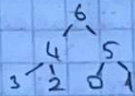
$$A[\text{parent}(i)] \leq A[i]$$

NOT: Bir ağacın heap tree olmasının ön koşulu bu ağacın bir complete binary tree olmasıdır.

Operations:

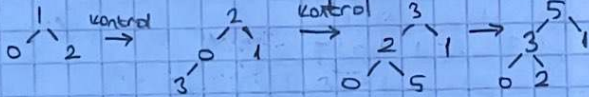
- insert()
 - delete()
 - get Max()
 - get Min()
- heapify() → ağacın özelliğinin korunması için

6 4 5 3 2 0 1



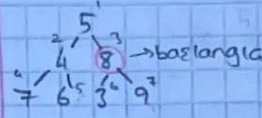
min-heap tree olması için
heapify() ile yer değişimi yapılır.

1 0 2 3 5 4 6 → insert



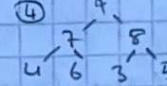
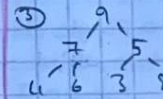
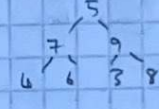
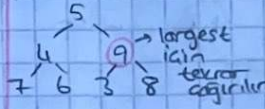
```
void max-heapify(int dizi[], int i, int N){
    int left = 2*i;
    int right = 2*i+1;
    if (left <= N && dizi[left] > dizi[i])
        largest = left;
    else
        largest = i;
    if (right <= N && dizi[right] > dizi[largest])
        largest = right;
    if (largest != i){
        swap(dizi[i], dizi[largest]);
        max-heapify(dizi, largest, N);
    }
}
```

- *Bu kod tek eleman için heapify yapar.
- *İzi açılığa da bakıp karıştırmaya yapıyor.
- *N elemanın N/2'nci elemanından başlamak en mantıklısı



① 9 > 8

② 7 > 4



```
void build-maxHeap(int dizi[]){
    int i;
    for(i=N/2; i>=1; i--){
        max-Heapify(dizi, i, N);
    }
}
```

Advantages of Heap:

- *Fast access to max/min element
- *Efficient insertion and deletion operations
- *Can be efficiently implemented on arrays.
- *Suitable for real-time applications

Disadvantages of Heap:

- *Not suitable for searching for an element other than max/min.
- *Extra memory overhead to maintain heap structure
- *Slower than other data structures like arrays and linked lists for non-priority queues.

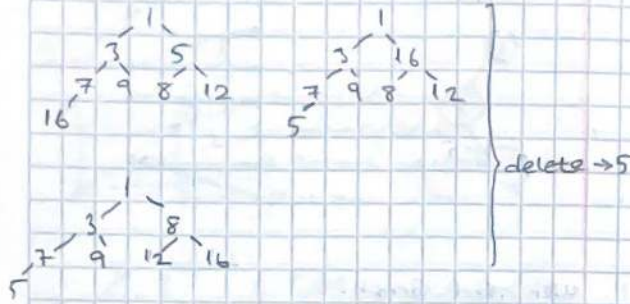
Applications of Heap Data Structures

1. Priority queues
2. Sorting Algorithm
3. Graph Algorithms (Dijkstra, Prim, A* search)
4. File Compression
5. Dynamic Programming
6. Load Balancing

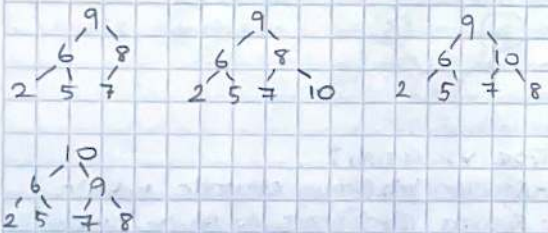
Delete → 9



*Delete yapılacak eleman sona atılır, dizide $N=N-1$ yapılır. Sonra heapify yapılır.



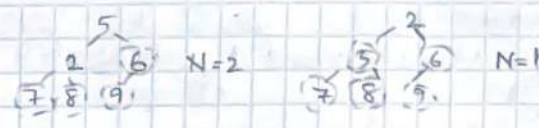
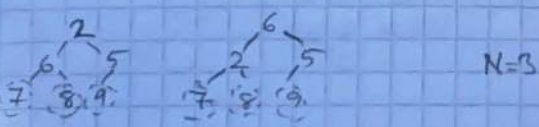
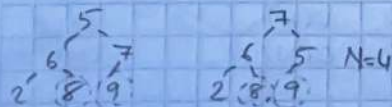
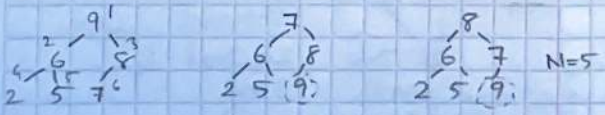
Insert → 10



NOT: önceki kod bozuk ağacın heapify işlemiydi. Ama insert işlemi zaten heap tree üzerinde olduğu için tüm elemanlara bakmamıza gerek yoktur.

HeapSort:

9 6 8 2 5 7



* $N-1$ adımda $O(N \log N)$ karmaşıklığında işlem tamamlandı.

NOT: Elemanlar eşit old zaman sıralamada pozisyon değiştirmiyorsa stable, değişiyorsa instable'dir.

*Heapsort bir instable algoritmadır. Bu bir dezavantajdır.

```
void heapSort(int arr[], int n){
    ht i;
    //build max heap
    for(i=n/2-1; i>=0; i--){
        heapify(arr, n, i);
    }
    for(i=n-1; i>0; i--){
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}
```

Ağacı karışık.
ken düzelt
max için

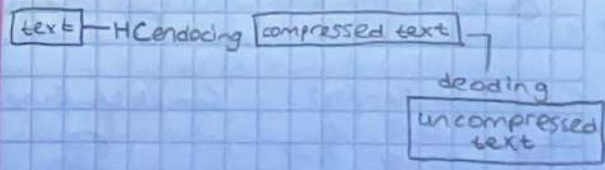
Priority Queue:

- Array (ordered)
- List (ordered)
- Array (unordered)
- Binary Heap Tree

implemente etme yöntemleri

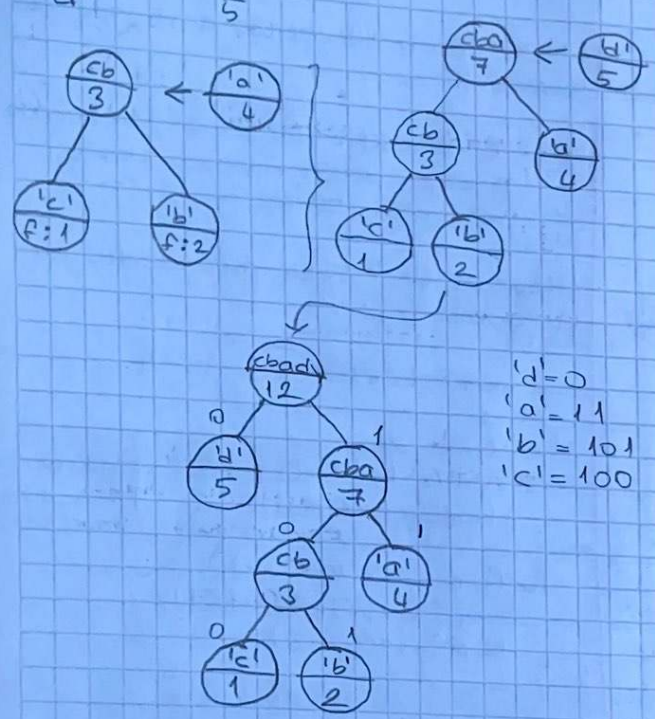
	insert	remove max pri.	find max pri.
ordered	$O(N)$	$O(1)$	$O(1)$
List	$O(N)$	$O(1)$	$O(1)$
unordered	$O(1)$	$O(N)$	$O(N)$
BHT	$O(\log N)$	$O(\log N)$	$O(1)$

Huffman Coding Using Priority Queue
(Lossless compression technique)

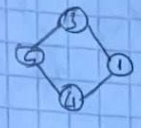


*Bir karakter diğerinin başında olmaz.

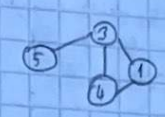
karakter	count	'c'	'b'	'a'	'd'
a	4	f:1	f:2	f:4	f:5
b	2				
c	1				
d	5				



Cycle Graph:



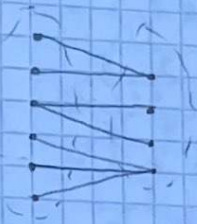
Cyclic Graph:



Directed Acyclic Graph:

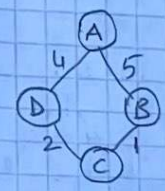


Bipartite Graph



Weighted Graph:

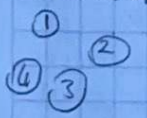
*yönlü / yönsüz olabilir.



Graphs:

- Non-linear data structure
- Vertex (node)
- Edge
- $G(V, E) \Leftrightarrow G(V, E)$

Null Graph:



No edges

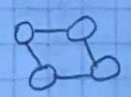
Trivial Graph:



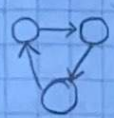
single vertex

* Çok fazla kenar yoksa list kullanmak daha mantıklıdır.

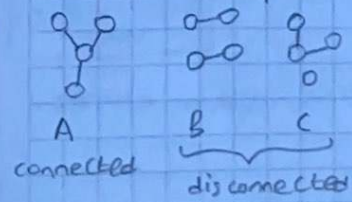
Undirected Graph:



Directed Graph:



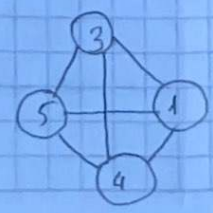
Connected Graph:



connected

disconnected

Complete Graph:



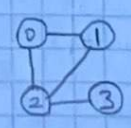
Tree vs. Graph:

- Ağaçlar grafların acyclic halidir.
- Grafta istediğimiz node yok olabilir.

Representation of Graphs:

- Adjacency list
- Adjacency matrix

ör:



0 → 1 → 2
1 → 0 → 2
2 → 0 → 1 → 3
3 → 2

List

Adjac. List

- Adding Edge $O(1)$
- Removing " $O(N)$
- Initializing $O(N)$

Adjac. Matrix

- $O(1)$
- $O(1)$
- $O(N \times N)$

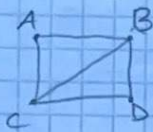
NOT: Dense graph:

Sparse Graph:

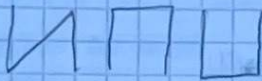
Operations on Graphs:

- Insert Node / Edges
- Delete Node / Edges
- Search
- Traverse (DFS, BFS)
 - ↳ katman/seviye öncelikli
 - ↳ derinlik öncelikli

Spanning Tree:



- *Acyclic
- *connected
- *Travelling sales-man problem v.b.



minimum spanning tree:

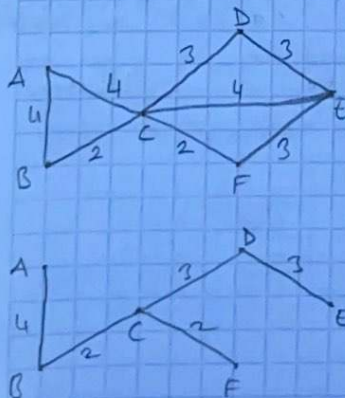
- Kruskal algorithm → en küçük sayıdan itibaren
- Prim's algorithm → baten sıradaki kenar döngü oluşturmaya-cak şekilde kenarlar seçilir. (greedy tech.)

*min. spanning tree 'de gidip geri dönme kısımları iki kez sayılmaz.

Formal Def. of Spanning trees:

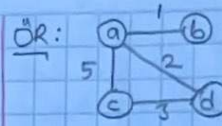
*A spanning tree is defined as a tree-like sub-graph of a connected undirected graph that includes all the vertices of the graph.

*MST has all the properties of a spanning tree with an added constraint of having the minimum possible weights among all possible spanning trees.

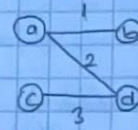


Greedy technique.

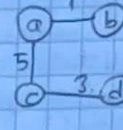
- BC - 2 ✓
- CF - 2 ✓
- CD - 3 ✓
- DE - 3 ✓
- FE - 3
- AB - 4 ✓
- AC - 4
- CE - 4



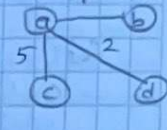
Graph



$w(T_1) = 6$
MST



$w(T_2) = 9$



$w(T_3) = 8$

Kruskal alg.

Disjoint set prob.

Union Find prob.

A	B	C	D	E	F
1	2	3	4	5	6

→ Başta hepsinin farklı setlerde old. gösterir.

Eğer iki kenar birleşirse o iki kenar aynı sette olur. (Aynı sayı yazılır.)

```
typedef struct {
    int u, v, w;
} EDGE;
```

```
typedef struct {
    EDGE edge_data[100];
    int n;
} EDGE_LIST;
```

```
void kruskal() {
    int [100], i, j, cno1, cno2;
    elist.n = 0;
```

```
for (i = 0; i < n; i++) {
    for (j = 0; j < i; j++) {
        if (Graph[i][j] != 0) {
```

node baş. ← elist.data[elist.n].u = i

node bit. ← elist.data[elist.n].v = j

node ağırl. ← elist.data[elist.n].w =

Graph[i][j];

```
        elist.n++;
    }
}
```

```
sort();
```



```

for(i=0; i<n; i++){
    belongs[i]=i;
}

```

```

spanlist.n = 0;

```

```

for(i=0; i<elist.n; i++){
    cno1 = find(belongs, elist.data[i].u);
    cno2 = find(belongs, elist.data[i].v);
    if(cno1 != cno2){
        spanlist.data[spanlist.n] = elist.data[i];
        spanlist.n = spanlist.n + 1;
        applyUnion(belongs, cno1, cno2);
    }
}

```

```

int find(int belongs[], int vertexno){
    return(belongs[vertexno]);
}

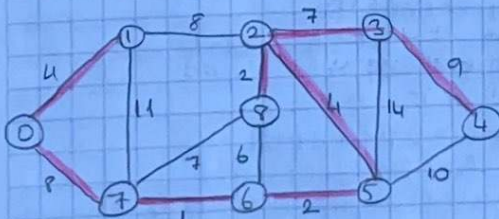
```

```

void applyUnion(int belongs[], int c1, int c2){
    int i;
    for(i=0; i<n; i++){
        if(belongs[i] == c2){
            belongs[i] = c1;
        }
    }
}

```

Prim's Algorithm:



- Önce başlangıç node'u seç
- 0 node'un komşularından en küçük olanı seç.
- 0 node ve komşusunun komşularından en küçük olanı seç.

1- 0 başlangıç
Sıradaki: 1-7

2- 1 sıradaki
Sıradaki: 2-7

3- 7 sıradaki
Sıradaki: 2-6-8

4- 6 sıradaki
Sıradaki: 2-8-5

5- 5 sıradaki
Sıradaki: 2-8-3-4

6- 2 sıradaki
Sıradaki: 8-3-4

7- 8 sıradaki
Sıradaki: 3-4

8- 3 sıradaki
Sıradaki: 4

9- 4 sıradaki

```

int main(){
    int no_edge;
    int selected[100];
    no_edge = 0;
    selected[0] = 1;
    int x, y;

```

```

while(no_edge < V-1){
    int min = 1000;
    x = 0;
    y = 0;
    for(i=0; i<V; i++){
        if(selected[i]){
            for(j=0; j<V; j++){
                if(!selected[j] && G[i][j]){
                    if(min > G[i][j]){
                        min = G[i][j];
                        x = i;
                        y = j;
                    }
                }
            }
        }
    }

```

```

    printf("%d - %d : %d\n", x, y, G[x][y]);
    selected[y] = 1;
    no_edge++;
}

```

```

return 0;

```


PRIM

- From any vertex
- It traverses one node more than one time

$$O(V^2), O(E \log V)$$

- It gives connected component as well as it works only on connected graph.

- Faster for dense graphs.

- Harder to implement

- Priority queue.

VS KRUSKAL

- From an edge with minimum weight.

- It traverses one node only once.

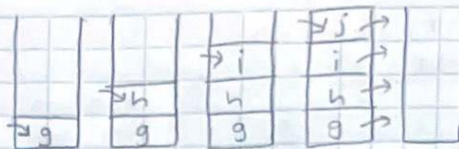
$$O(E \log V)$$

- It can generate forest at any instant as well as it can work on disconnected components.

- Faster for sparse graphs.

- Easier to implement

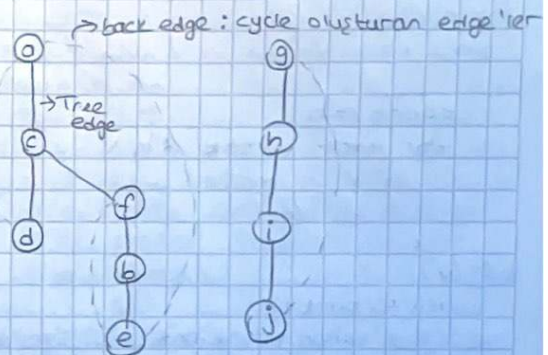
- Disjoint set.



a c d f b e g h i j

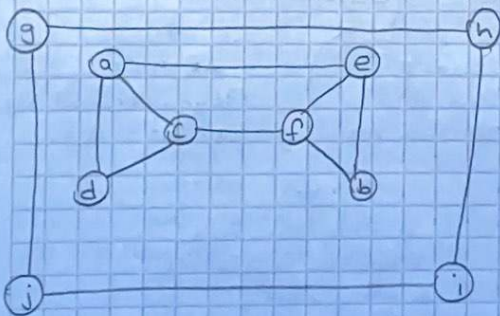
a_{1,16}
c_{2,5}
d_{3,1}
f_{4,14}
b_{5,3}

e_{6,12}
g_{7,10}
h_{8,9}
i_{9,8}
j_{10,17}



Traversal on Graphs

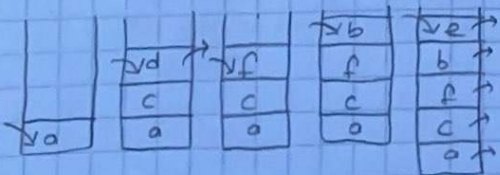
- Depth-First-Search (DFS)
- Breath-First-Search (BFS)



2 connected components

a-c-d-f-b-e-g-h-i-j DFS

*stack ile implemente etmesi kolaydır.



Pseudo Code :

// input Graph $G = \langle V, E \rangle$

mark each vertex in V with 0 as a mark of being unvisited

count $\leftarrow 0$

for each vertex in V do

if v is marked with 0

dfs(v)

dfs(v)

count $\leftarrow$ count + 1

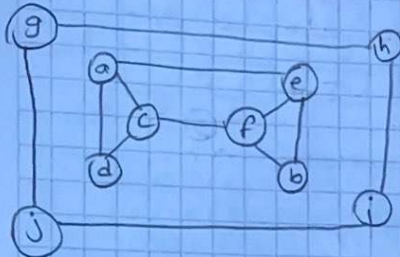
mark v with count.

for each vertex w in V adjacent to v do

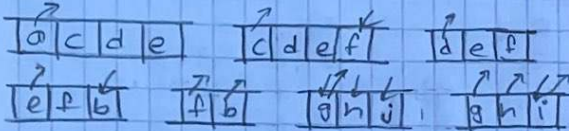
if w is marked with 0

dfs(w)

Breadth-First Search:



* queue ile
imp. edilir.



Algoritma: BFS(a)

mark each vertex in V with $\emptyset$ as a mark
of being unvisited

count $\leftarrow 0$

for each vertex v in V do

if v is marked with $\emptyset$
BFS(v)

BFS(v)

count $\leftarrow$ count + 1

mark v with count and initialize a queue
with v .

while the queue is not empty do

for each vertex w in V adjacent to
the front vertex do

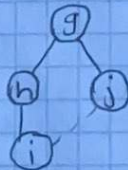
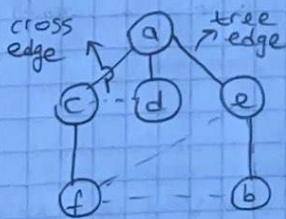
if w is marked with $\emptyset$

count $\leftarrow$ count + 1

mark w with count

add w to the queue

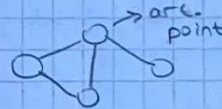
remove the front vertex from the queue



a₁ c₂ d₃ e₄ f₅ b₆ g₇ h₈ i₉ j₁₀

	DFS	BFS
Data struc.	Stack	queue
Number of vertex orderings	Two ordering	One ordering
Edge types	Tree and back edges	Tree and cross edges
Efficiency for adj. matrix	$\Theta(V^2)$	$\Theta(V^2)$
Efficiency for adj. list	$\Theta(V+E)$	$\Theta(V+E)$

Articulation point: Grafin bir noktada
ayrıldığında kalan grafların ayrı subgraf
old. noktadır.



Shortest Path Algorithms

Single source

- Dijkstra Alg.

- Bellman-Ford Alg.

All pairs

- Floyd-Warshall Alg.

- Johnson's Alg.

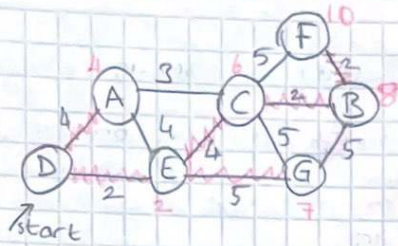
Topological Sort

* Bağlılık ilişkilerini
de dahil eder.

- Decrease and conquer

- DFS

NOT: Negatif ağırlık old. zaman Dijkstra
çalışmaz.



- 1- Başlangıç hariç tüm düğümlere uzaklık başta infinity olur.
- 2- D'den E'ye 2, A'ya 4 var. E seçilir.
- 3- A'ya D'den gitmek E'den gitmekten daha az maliyetli. AD yolu seçilir.
- 4- E'den C ve G'ye gidilir.
- 5- A'nın komşularından daha az yol yok.
- 6- C'den B'ye gidilir, C'den F'ye gitmek C-B-F yapmaktan daha maliyetli.
- 7- B'den F'ye gidilir.

PSEUDO CODE FOR DIJKSTRA

- Set initial distances for all vertices: ∞ for source, infinity for all the other vertices
- Choose the unvisited vertex with the shortest distance from the start to be the current vertex. So the alg. will always start with the source as the current vertex
- For each of the current vertex's unvisited neighbor vertices, calculate the distance from the source and update the distance if the new calculated distance is lower.
- We are now done with the current vertex, so we mark it as visited. A visited vertex is not checked again.
- Go back to step 2 to choose a new current vertex, and keep repeating these steps until all vertices are visited.
- In the end we are left with the shortest path from the source vertex to every other vertex in graph

Pattern Matching

*T metni içinde verilen bir P pattern'ı bulunurken

- n ve m $\rightarrow$ P ve T'nin uzunlukları
- $\Sigma \rightarrow$ alfabet
- $P_i \rightarrow$ Pattern'in i'inci harfi
- a, b, c single letters in Σ
- x, y, z strings

Brute Force

* ilk harften başlayarak sağa doğru tüm olasılıkların denenmesidir.

worst case: hiç olmaması veya son harf hariç hepsinin eşleşiyor olmasıdır.
 $O(n*m)$

best case: patternin en başta olması
 $O(n)$

NOT: tekrar eden karakterli pattern'lerde pattern boyutu kadar kaydıramayız.

Boyer-Moore

Bad symbol shift
Good suffix shift

* Pattern'de hiç olmayan karakter bulunur, so o karakterden sonradan bakılır.

* Pattern'de olan karakterde olan kısmı alt alta gelecek şekilde hizalanır.

* Pattern'de birden fazla olan harflerde eşleşen harften bir önceki ile match edilir.

S E R
B A R B E R $t_i(s) - 2 = 4$ kaydırılır.

S E R
B A R B E R

A E R
B A R B E R $t_i(A) - 2 = 4 - 2 = 2$

A E R
B A R B E R

kaydırılır.

$$d_1 = \max\{t_1(c) - k, 1\}$$

hangisi daha büyükse o kadar shift

A A R B E R
B A R B E R

$$t_1(A) - k = -1$$

4 5

Good suffix

k	Pattern	$d_2 \rightarrow$ kaydırma miktarı
1	ABCBAB	2
2	ABCBAB	4
3	ABCBAB	4
4	ABCBAB	4
5	ABCBAB	4

* Tekrar eden kısımlar içeren pattern'lerde faydalıdır.

$$d = \begin{cases} d_1, & k=0 \\ \max\{d_1, d_2\}, & k>0 \end{cases}$$

* Harf kelimede var mı? kag

NOT: Bulunamayan durumda $O(m/n)$ worst case'de karmaşıklığı olur.

Horspool:

- * karakter pattern'de yoksa sonrasına kayar.
- * varsa sağdan ilk o harfin olduğu yere kayar.

DER } DER
LEADER } LEADER

SER } SER
BARBER } BARBER

NOT: Pattern'in son karakteri önemli

Rabin Karp

* Text ve pattern'in hash değeri çıkarılır.

* Pattern'in hash değeri ile pattern'de karşılaştırdığımız text parçasının hash değeri aynı ise pattern'de karşılaştırılır.

$$26535 \rightarrow 613$$

% 997

$$3141592653589793$$

$$31415 \rightarrow 508$$

$$14159 \rightarrow 201$$

$$41592 \rightarrow 715$$

$$26535 \rightarrow 613$$